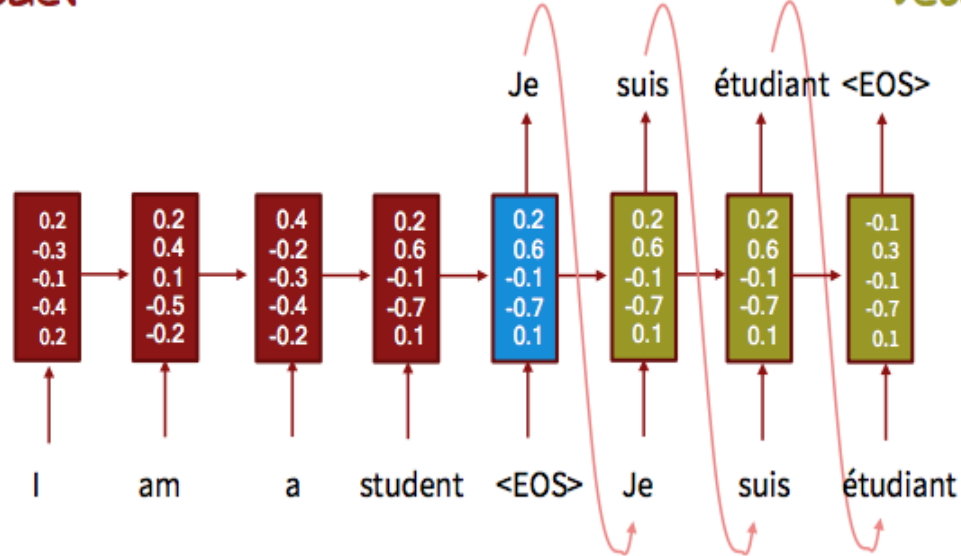
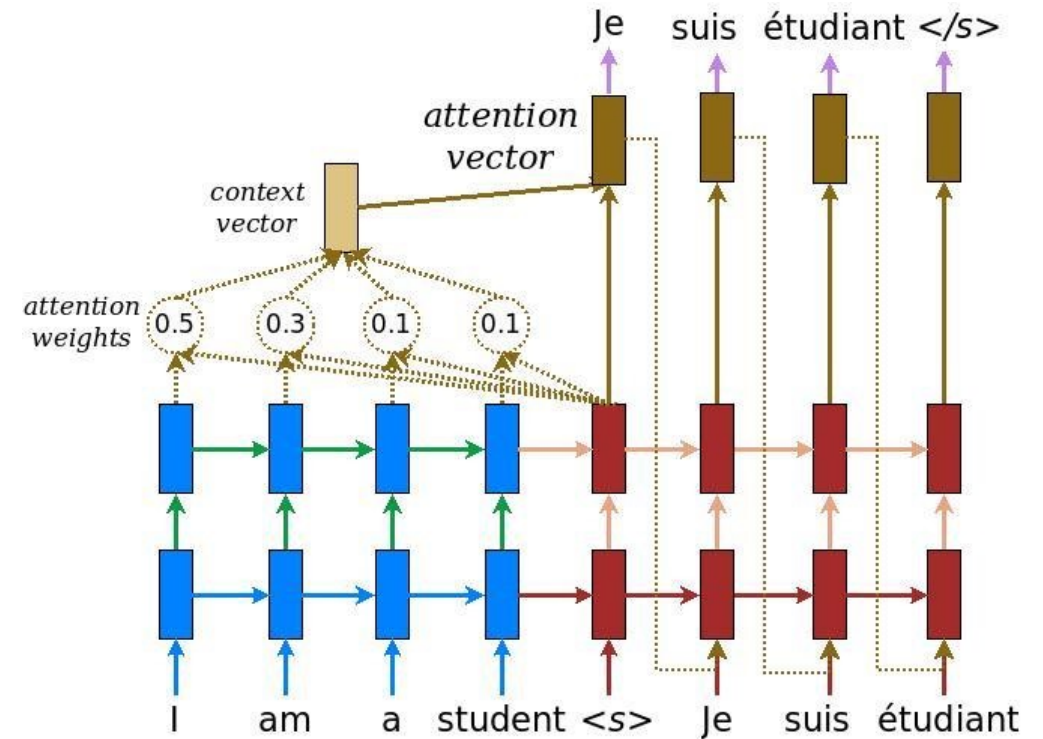
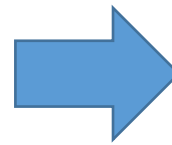


RNN+Attention

Encoder



$$h_t = \tanh(W[x_t] + Uh_{t-1} + b)$$



RNN+Attention

- Attention is a mechanism for making specific parts of the input more accessible to the decoder
- Instead of the decoder only receiving the summary vector for the input (the last hidden state), using attention it receives a linear combination of the hidden states of the tokens of the sequence that are most relevant for the next decoded word
 - A form of a soft alignment
- The coefficients are themselves dynamically computed
 - We will define this formally in the next slides

The Transformer Architecture

Attention and Transformers

- Attention: a way to model an implicit soft alignment between the words in the source sentence and the words in the target
 - Helps “select” what words in the source are most important for generating the next word
 - Inspired by human “attention:” limited in its capacity
- More generally, the same mechanism can be used for defining context as a distribution over tokens
 - We can also think of the previous layer as context for the next layer: this is called ‘self-attention’

Self-Attention

- Self-attention is the use of attention to generate, from a sequence of word embeddings, a new sequence of word embeddings whose representation is some aggregation of the word embeddings in the previous layer
- Why would we want to do that?

Attention Visualizations

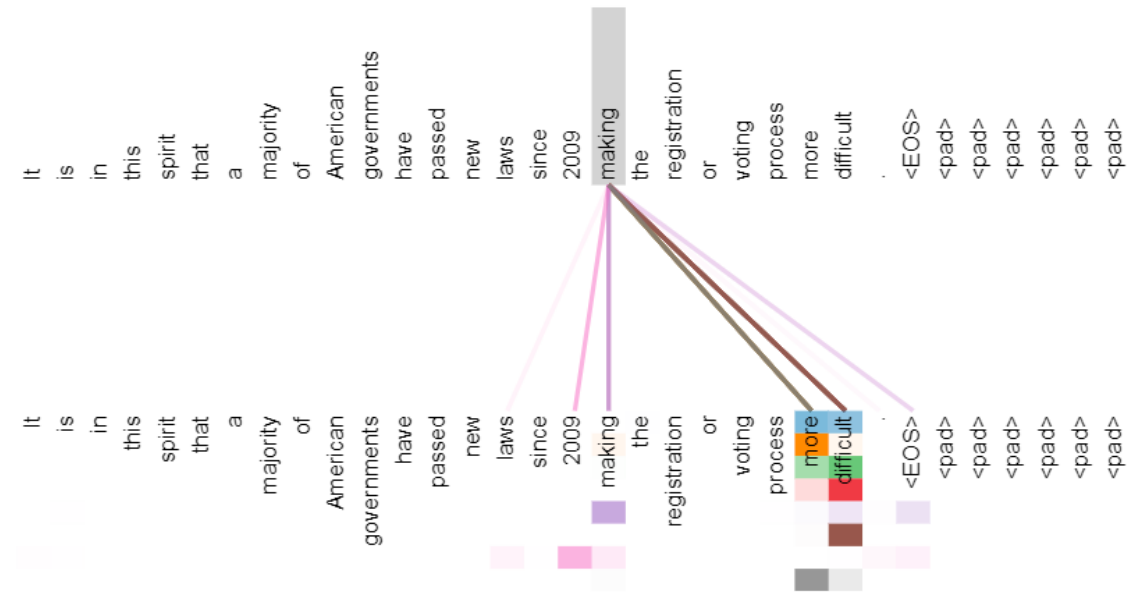
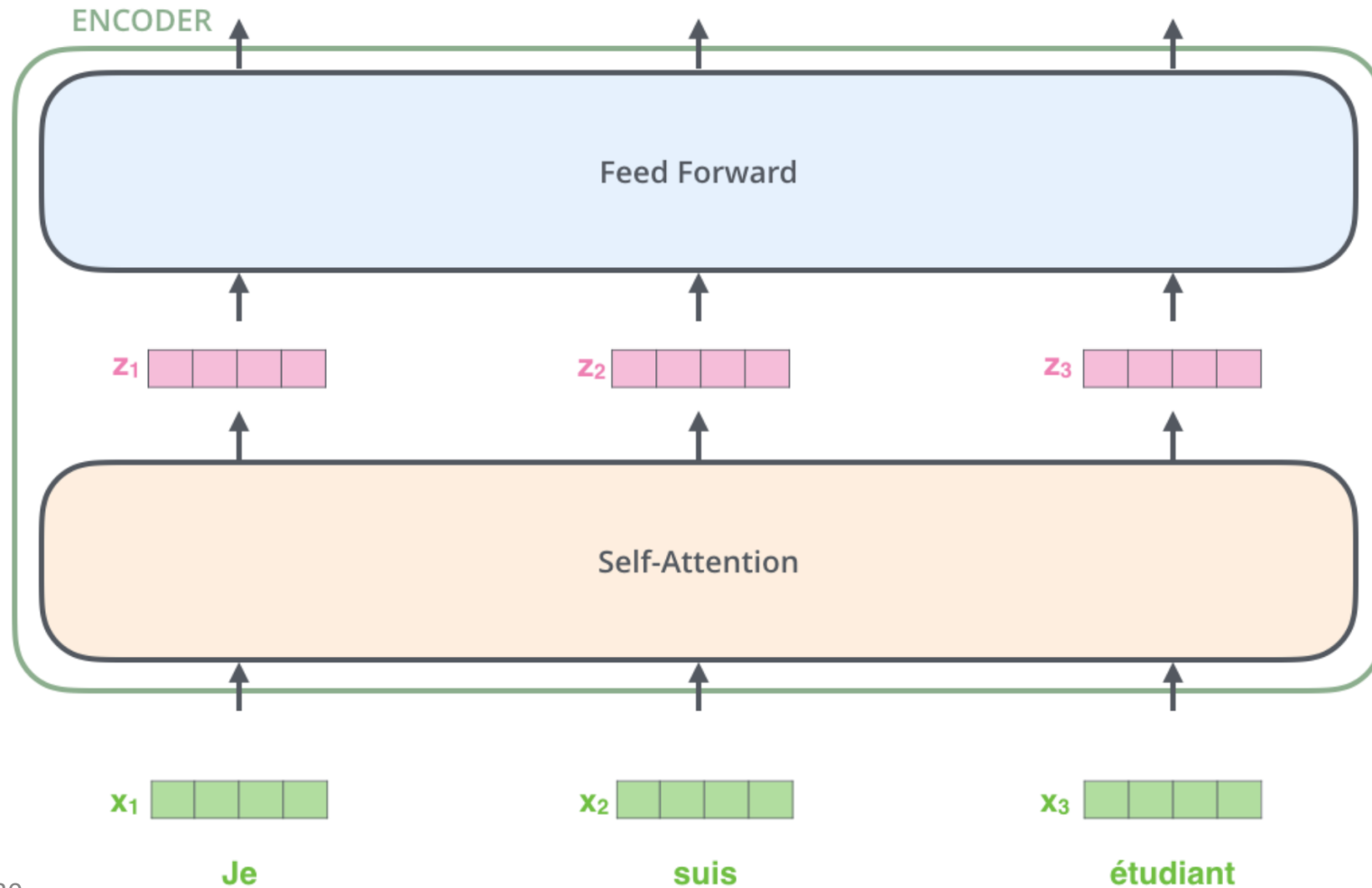


Figure 3: An example of the attention mechanism following long-distance dependencies in the encoder self-attention in layer 5 of 6. Many of the attention heads attend to a distant dependency of the verb 'making', completing the phrase 'making...more difficult'. Attentions here shown only for the word 'making'. Different colors represent different heads. Best viewed in color.

Self-Attention

- Traditional ('non-contextualized') embeddings of a word represent the word form over all its different senses
 - For example, the word *make* could be used in many different senses (e.g., להצחיק — make ... laugh, להקשות — make difficult, להבין — make a cake, make a dog's breakfast of ... - לעשות משהו ברישול)
 - The context of a word may disambiguate it, resulting in a representation that is tailored to the sentence it appears in
- Self-attention begins with embedding every word according to its form, and an embedding of its position
- It then iteratively constructs a representation of the word by self-attending to the representations in the previous layer

The Self-Attention Mechanism



The Illustrated Transformer

<https://jalammar.github.io/illustrated-transformer/>
[Jay Alammar](#)

The Illustrated Transformer

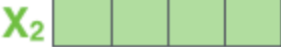
<https://jalammar.github.io/illustrated-transformer/>
[Jay Alammar](#)

Input

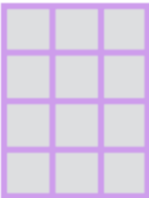
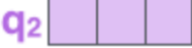
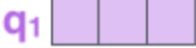
Thinking

Machines

Embedding

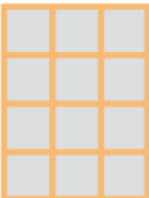
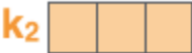
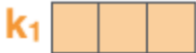


Queries



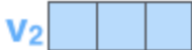
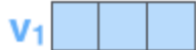
W^Q

Keys



W^K

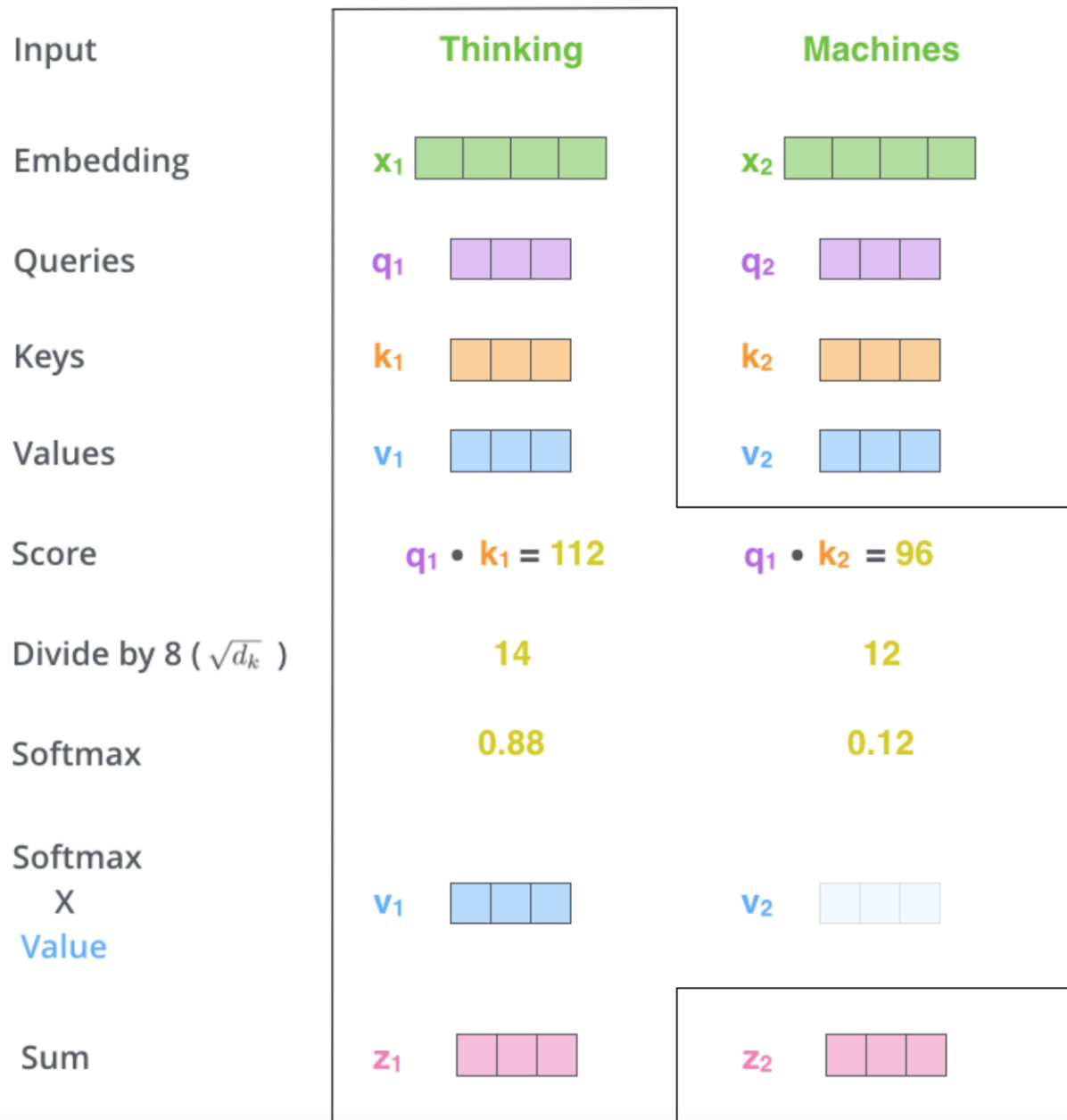
Values



W^V

Multiplying x_1 by the W^Q weight matrix produces q_1 , the "query" vector associated with that word. We end up creating a "query", a "key", and a "value" projection of each word in the input sentence.

d_k : the dimension
of the keys/values



The Illustrated Transformer

<https://jalammar.github.io/illustrated-transformer/>

[Jay Alammar](#)

Attention: Formal Version

Given an input vector series $(x_1, \dots, x_n) \in \mathcal{R}^d$, we may write it as a matrix $X \in \mathcal{R}^{d \times n}$. Given the attention's parameters $W^K, W^Q \in \mathcal{R}^{d_k \times d}$ and $W^V \in \mathcal{R}^{d_v \times d}$. Compute:

$$\begin{aligned} Q &= W^Q \cdot X = \begin{pmatrix} | & | & \cdots & | \\ q_1 & q_2 & \cdots & q_n \\ | & | & \cdots & | \end{pmatrix} \\ K &= W^K \cdot X = \begin{pmatrix} | & | & \cdots & | \\ k_1 & k_2 & \cdots & k_n \\ | & | & \cdots & | \end{pmatrix} \\ V &= W^V \cdot X = \begin{pmatrix} | & | & \cdots & | \\ v_1 & v_2 & \cdots & v_n \\ | & | & \cdots & | \end{pmatrix} \end{aligned}$$

A self-attention operation given Q, K, V returns a matrix $Z \in \mathcal{R}^{d_v \times n}$ of the following form:

$$Z = \begin{pmatrix} | & | & \cdots & | \\ z_1 & z_2 & \cdots & z_n \\ | & | & \cdots & | \end{pmatrix}$$

where

$$z_i = \sum_{j=1}^n sm \left(\frac{q_i \cdot k_j}{\sqrt{d_k}} \right) \cdot v_j$$

where sm is the softmax over $\left\{ \frac{q_i \cdot k_j}{\sqrt{d_k}} \right\}_{j=1}^n$

Multi-head Attention

- Transformer networks often use multi-head attention layers
- This means that a fixed number of attention mechanisms (“heads”)
 - Number of heads is usually one digit or low two digits
- Eventually, the output of the heads is concatenated to form the input to the next layer, and multiplied by an additional project matrix

Multi-head Attention

- Formally:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

Residual Connections

- Residual connections effectively create a bypass around the module they go around (either a feed-forward or an attention)
- Assume $\text{SubLayer}(x)$ is the computation of the module, then after adding a residual connection, the layer becomes:

$$\text{Add\&Norm}(x) = \text{LayerNorm}(x + \text{SubLayer}(x))$$

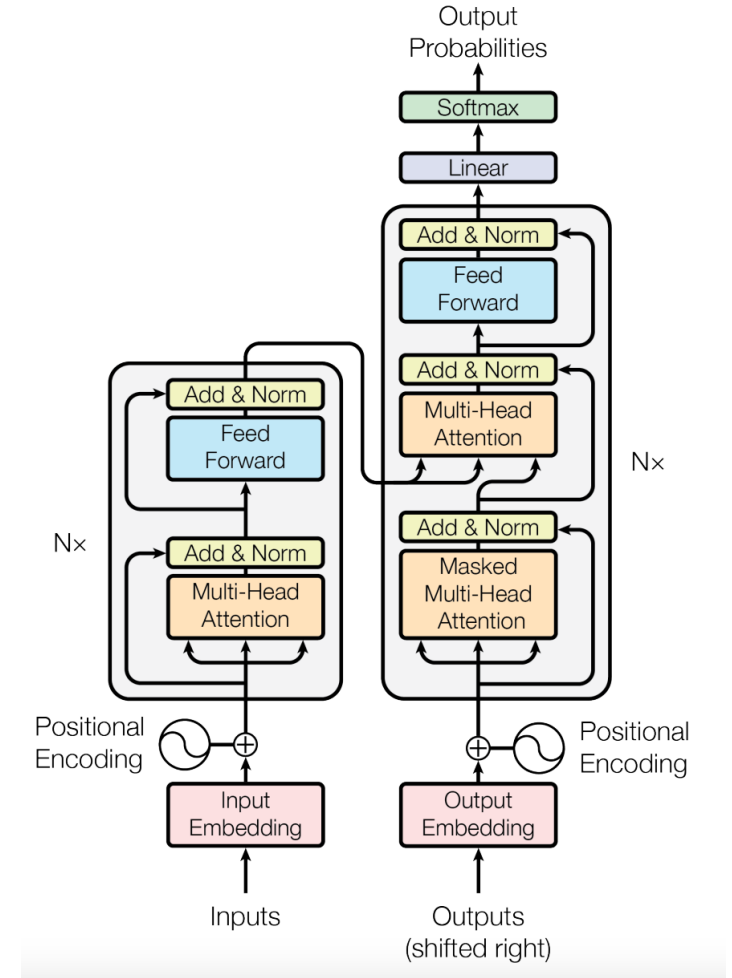
- LayerNorm is a mechanism for increasing the stability of the network (Ba et al., 2016)

The Transformer Model

- The Transformer (Vaswani et al. 2017) model is the standard neural model used in NLP today
 - Although it does have several important variants
- It is often used as a sequence-to-sequence model, but is not an RNN
 - This means that like the RNN encoder-decoder, it is essentially a model for predicting the next word, given the input sentence and prefix

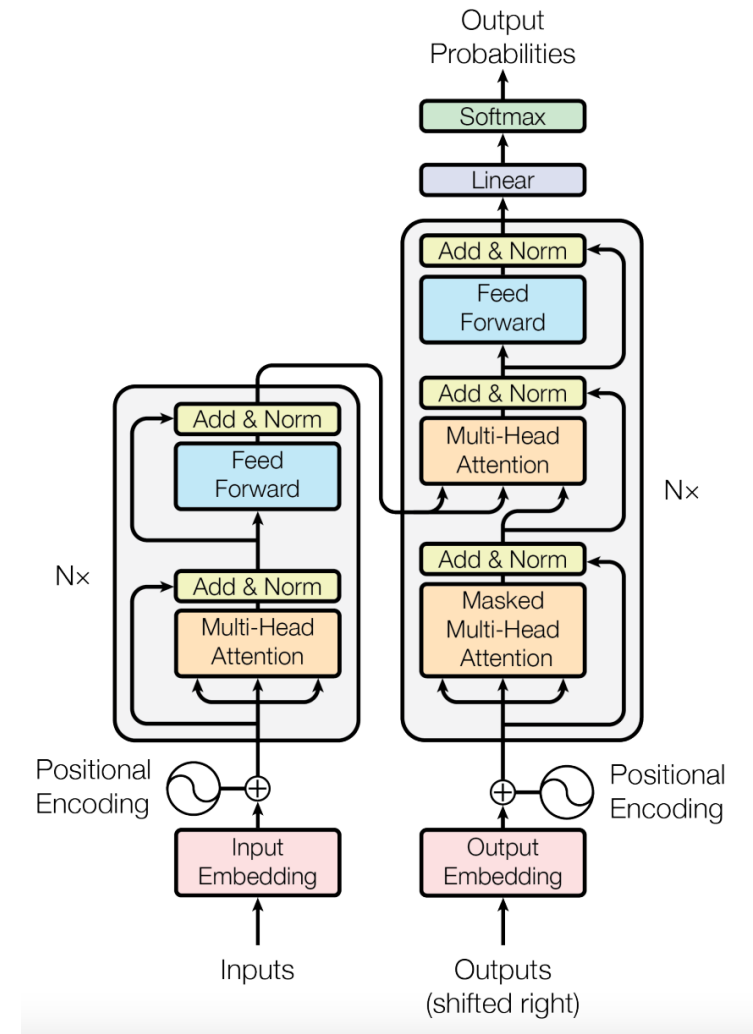
$$p_{\theta}(y_i | y_{i-1}, \dots, y_1, \mathbf{x})$$

- Unlike in RNNs, the input size is fixed, and special padding tokens are used to reach uniform input length



The Transformer Model

- The Transformer model is divided into blocks. Each block receives a representation of the tokens in the previous layer as vectors in and computes a representation of the next layer
- The network is both deep (often double-digit number of layers in the encoder and the decoder) and wide

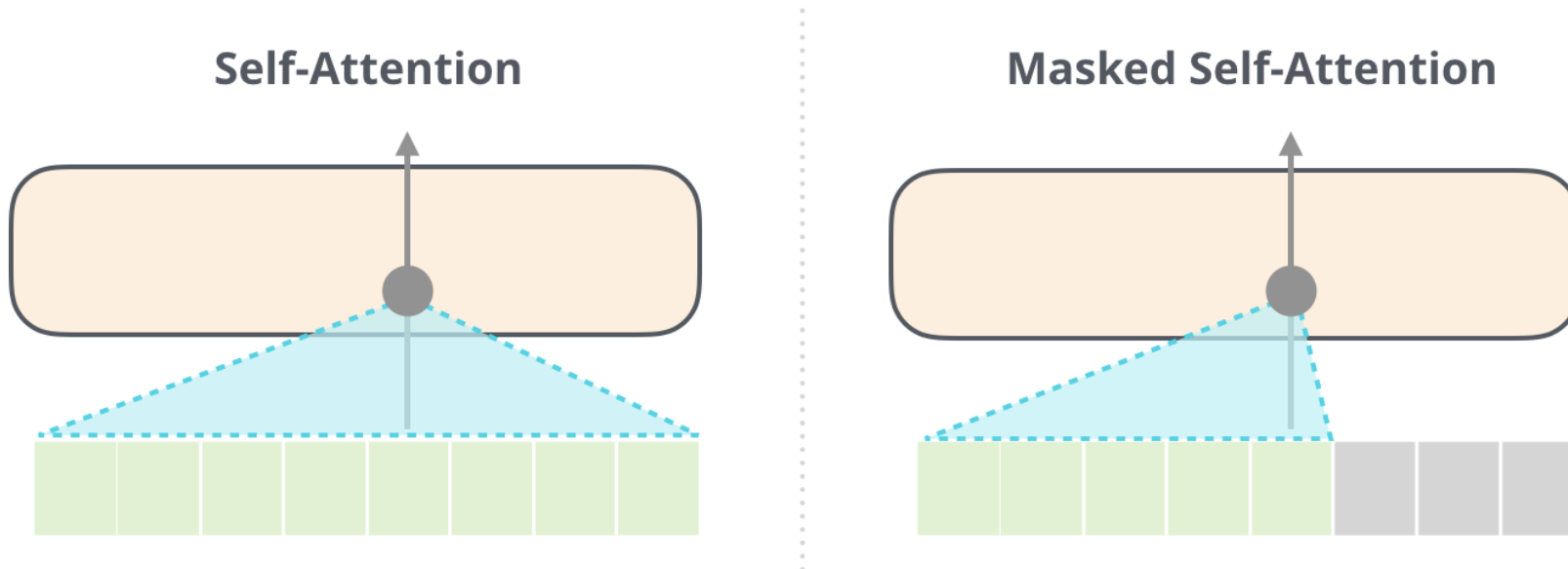


Decoder: Cross-Attention

- The decoder produces a distribution over the next word given a prefix
- To do so, it needs to attend both to the output of the encoder and to the encoding of the tokens in the prefix
- Cross-attention is the case where the queries Q come from one source and the keys K come from another
 - Otherwise, the attention mechanism stays the same
 - In the case of the transformer, queries come from the concatenation of the encoder's output and the output of the "masked multi-head attention"

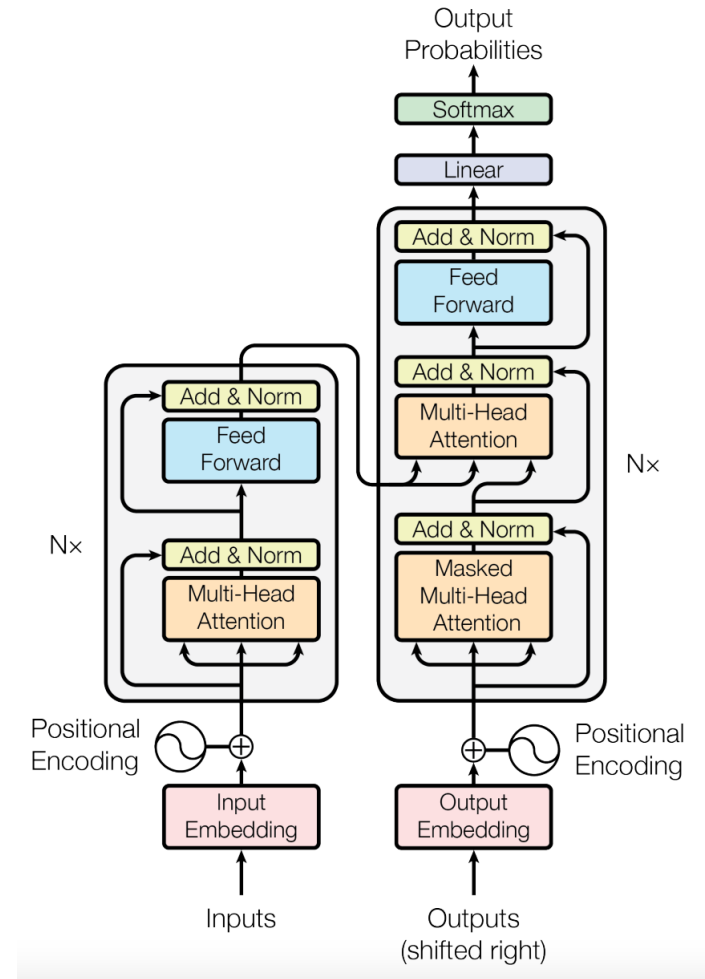
Decoder: Masked Attention

- In order to save compute, the decoder caches the previously computed activations
 - This means that the activations in position i may not depend on indices $j > i$



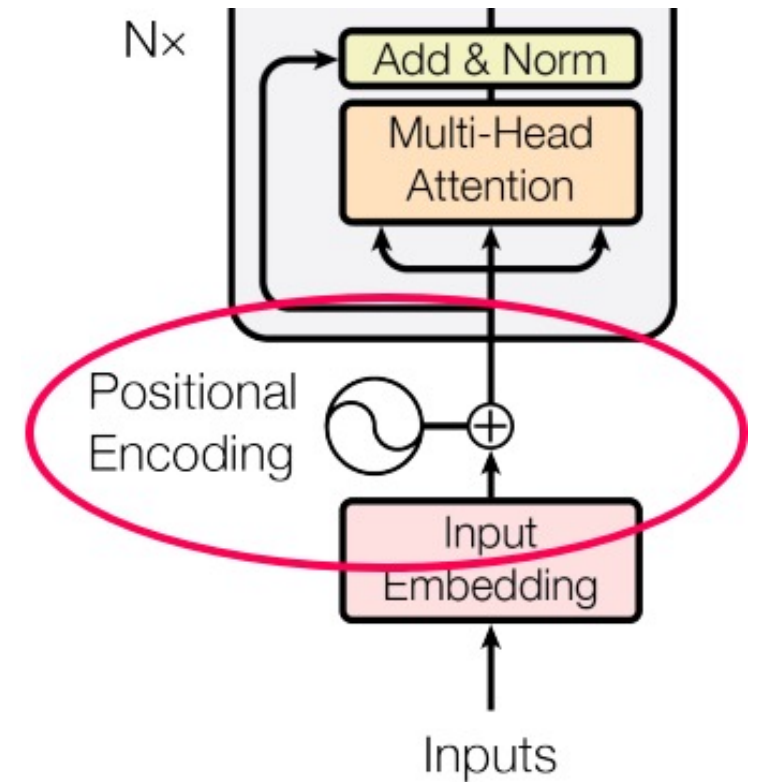
The Transformer Model

- The decoder for the model is similar in architecture to the encoder, but with several important differences:
 - Each decoder block includes a block of multi-head self-attention, followed by a block of attention over both the encoder input and the output of the self-attention
 - The input to the decoder is the prefix
 - The decoder outputs a probability over the possible next tokens



The Encoding of Positions in Transformers

- Tokens are represented in the Transformers as a concatenation of an input embedding (non-contextualized) and a positional embedding (different embedding for every position index)
- The positional embeddings are the only elements that break the symmetry between tokens
- Positional embeddings are randomly initialized and learned during training
- The model can therefore learn to take into consideration any patterns in the relative positions, as long as it is within the context window



Training and Decoding with the Transformer

- The Transformer is trained with standard log probability loss, similar to the ones used for neural language models

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N \log[p_{\theta}(y_i | y_{i-1}, \dots, y_1, \mathbf{x})]$$

- Unlike RNNs, the Transformer does not require backpropogating through time
 - It is essentially a feed-forward network
 - Allows more parallelization in training
- The text is divided into segments using a sliding window (often with overlaps)
- Inference is essentially the same: can be done greedily, through beam search or using more advanced decoding techniques
 - Suffers from the same exposure bias...

Advantages of Transformers

- Transformers have shown superior results over RNNs in many NLP tasks, including Machine Translation
- Much easier to train than RNNs (no backpropagation through time)
- Less sensitivity to position
 - The input is given in parallel and not sequentially
 - Positional embeddings are learned